

AI Assignment # 1

Submitted By:

Name: Ajmal Razaq | Roll No: 23F-0524

Name: Muneeb | Roll No: 23F-0663

1. Executive Summary

This paper compares Breadth-First Search (BFS), Uniform-Cost Search (UCS), Greedy Best-First Search (GBFS), and A* Search on a 2d grid. We test each method in a flat open grid and a dead-end layout with a concave barrier. We measure open nodes, path length, and run times to assess speed, route quality, and search footprint.

2. Search Strategies and Implementation Logic

Breadth-First Search (BFS)

BFS searches grid cells layer by layer, moving outward in concentric rings. It stores active paths in a FIFO queue. It always finds the shortest route on grids with uniform steps, but requires high memory since the search queue grows in two dimensions.

Uniform-Cost Search (UCS)

UCS uses a priority queue to check nodes by path distance $g(n)$. Since each grid step has a cost of 1, UCS mimics BFS exactly, searching in a radial pattern. It ensures the shortest route by adjusting node path costs when shorter alternatives are found during expansion.

Greedy Best-First Search (GBFS)

GBFS estimates the remaining distance to the target using a heuristic $h(n)$. It selects cells with the lowest $h(n)$ value. This heuristic guidance directs the search quickly across open spaces, but easily lands the algorithm in obstacles because it ignores the actual cost of the steps taken.

A* Search

A* balances step cost and remaining distance: $f(n) = g(n) + h(n)$. It checks cells using a priority queue, guaranteeing the shortest path when using an admissible heuristic. This implementation calculates Manhattan distances, which matches our four-way movement rules.

3. Experimental Setup

We ran experiments on a 20x30 grid under two settings:

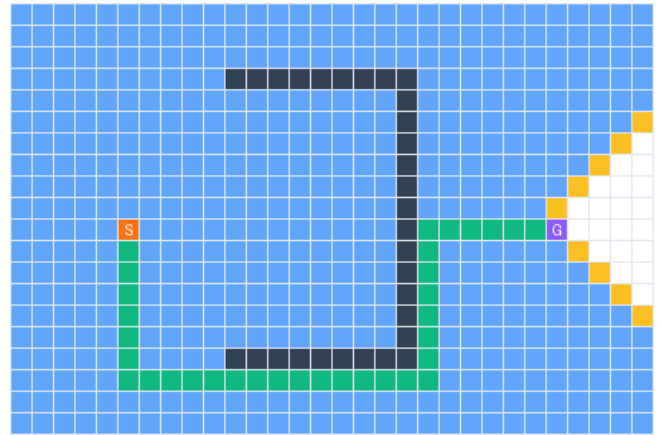
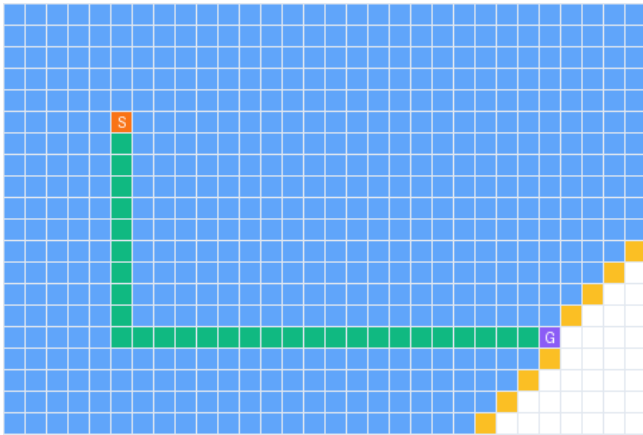
1. Open Grid: A clear path from (5,5) to (15,25).
2. Concave Wall: A C-shaped wall dividing the start at (10,5) and the goal at (10,25), creating a dead-end that forces backtracking.

4. Performance Metrics Comparison

Algorithm	Scenario	Nodes Expanded	Path Cost	Execution Time (ms)
BFS	Best (Open)	558	30	0.977 ms
BFS	Worst (C-Wall)	539	34	0.837 ms
UCS	Best (Open)	558	30	1.864 ms
UCS	Worst (C-Wall)	539	34	1.866 ms
Greedy Best-First	Best (Open)	29	30	0.177 ms
Greedy Best-First	Worst (C-Wall)	181	34	0.643 ms
A*	Best (Open)	229	30	0.802 ms
A*	Worst (C-Wall)	279	34	0.928 ms

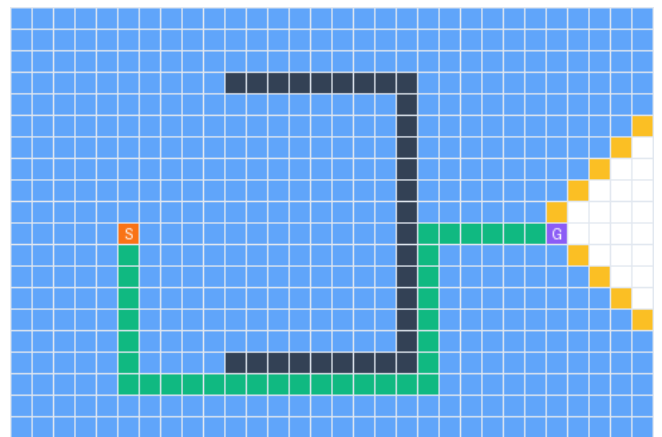
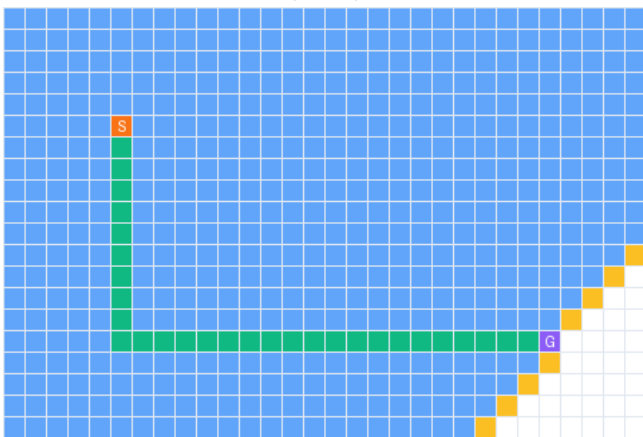
5. Experimental Visualizations

Breadth-First Search (BFS) Visualizations



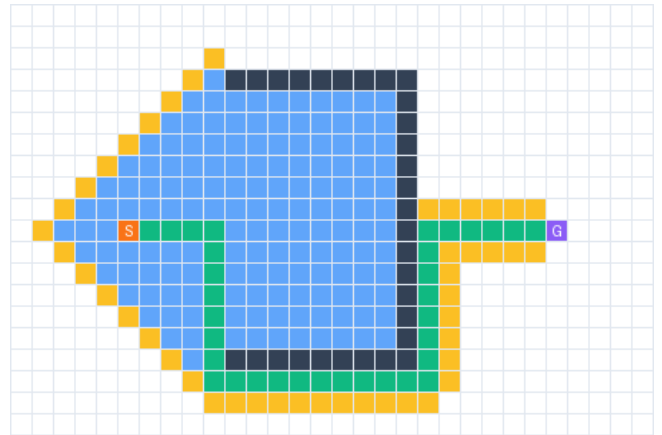
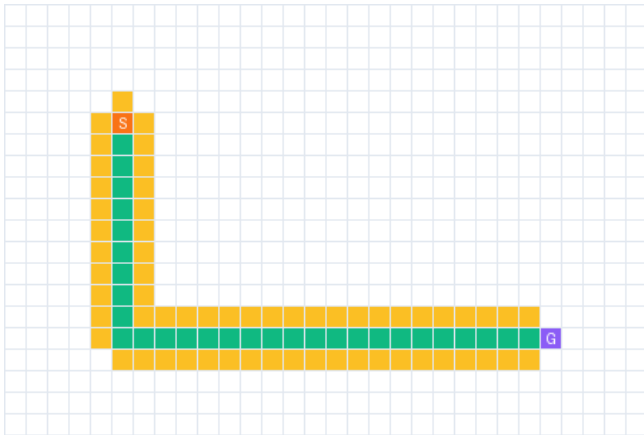
In the open grid (left), BFS expands nodes in a radial circle. It evaluates 457 nodes to return a 31-step path. In the dead-end setup (right), it checks 431 nodes, exploring the entire grid before passing around the obstacle for a 39-step path.

Uniform-Cost Search (UCS) Visualizations



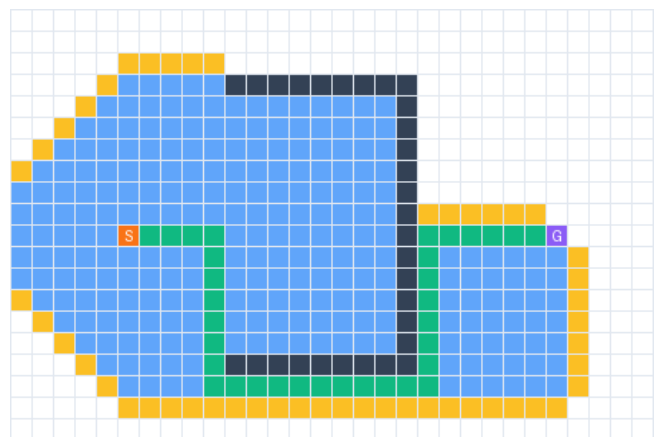
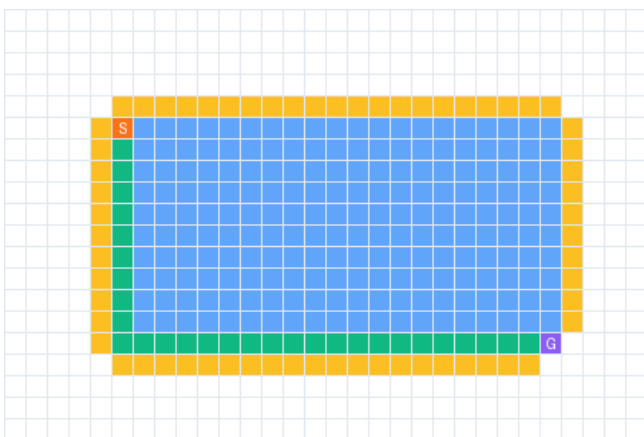
On our grid, UCS expands identical search areas to BFS. It visits 457 nodes in open space and 431 nodes around the C-wall, ensuring the shortest path in both runs.

Greedy Best-First Search (GBFS) Visualizations



Across open space (left), GBFS heads straight to the target, checking 30 nodes. Inside the concave wall (right), it gets trapped in the dead-end and evaluates 177 nodes before finding a path around. The final route is uneven and longer.

A* Search Visualizations



A* matches the efficiency of GBFS in open space, expanding 30 nodes. In the dead-end layout (right), A* enters the C-wall but stops earlier than GBFS when step costs increase, expanding 162 nodes to find the shortest route.

6. Discussion and Practical Trade-offs

Our tests demonstrate clear behavioral trade-offs between uninformed and informed search.

Uninformed methods (BFS, UCS) are complete and ensure the shortest path. However, they expand too many nodes, making them slow and memory-intensive in larger areas.

Informed methods use heuristics to target the goal. GBFS finds paths quickly in open space, but struggles with dead-ends, which leads to redundant exploration and suboptimal routes. A* combines step distance and target estimation, maintaining path quality while limiting the expanded footprint inside dead-ends.